

Transactions — Quick Revision Summary

1. Database Transaction

A group of DB operations treated as **one single unit**. Even if it's multiple SQL statements, business-wise it's one operation that must fully succeed or fully fail.

2. ACID Properties (Overview)

4 guarantees for safe transactions: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

2.1 Atomicity

All steps happen together, or none happen. No partial updates (e.g., money can't leave one account without reaching the other).

2.2 Consistency

Transaction must move DB from one **valid state** to another valid state (business rules must never be violated, e.g., balance can't go negative).

2.3 Isolation

Controls what one transaction can "see" while another is running at the same time, so concurrent transactions don't mess up each other's data.

2.4 Durability

Once committed, data is permanent — survives crashes, restarts, or power loss.

3. ACID as One Combined Guarantee

All 4 properties together answer: *"Can DB safely move from one valid state to another, even with failures & multiple transactions running together?"*

4. Transaction Boundary

Defines **where a transaction starts and ends** — usually a service method (like `transfer()`) represents one complete business transaction.

5. Begin, Commit, Rollback

- **Begin** → starts transaction (autocommit off).
- **Commit** → save changes permanently.
- **Rollback** → undo/discard changes if something fails.

6. Transaction Layers (JDBC → JPA → Hibernate → Spring)

All 4 are just different **layers/APIs** controlling the *same* underlying DB transaction:

- **JDBC**: lowest level, manual begin/commit/rollback, developer handles everything.
- **JPA**: uses `EntityManager`, hides JDBC details but still has boilerplate.

- **Hibernate:** own `Session`/`Transaction` API, internally uses JDBC.
- **Spring:** removes all this boilerplate — just handles it automatically.

7. Spring's `@Transactional`

Annotation telling Spring: "run this method/class inside a managed transaction."

- **Method-level** → only that method is transactional.
- **Class-level** → applies to all public methods (can be overridden per method).
- Usually placed in the **service layer** since it represents one full business use case.

8. Complete Mental Model

Flow: Caller calls method → Spring starts/joins transaction → repo operations run inside it → success = commit, failure (rollback-eligible exception) = rollback.

9. Key Takeaways (One-liner Recap)

- Transaction = one logical unit of work.
- Atomicity = all or nothing.
- Consistency = valid state to valid state.
- Isolation = safe concurrent execution.
- Durability = committed data survives failures.
- Begin/Commit/Rollback = transaction lifecycle.
- JDBC/JPA/Hibernate = different APIs, same underlying DB transaction.
- Spring's `@Transactional` = automates the whole process, mostly used in service layer.